

LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Dosen Pengampu:

1. Agus Ambarwati,S.Pd., M.Kom.
2. Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM., ASEAN Eng.

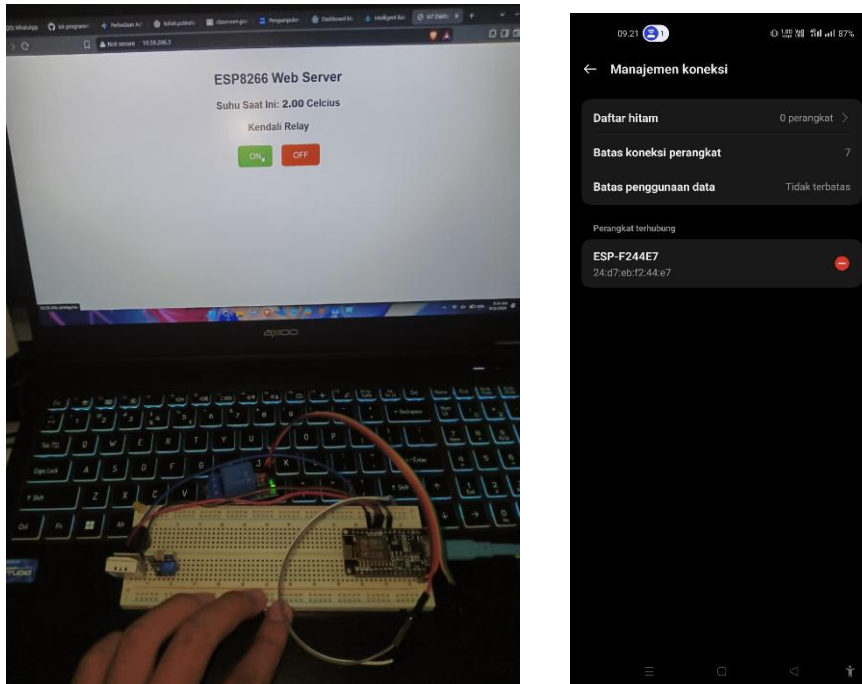


Disusun Oleh:

Machmud Al Hasan (NPM 25782044)

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
TAHUN 2026

Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)

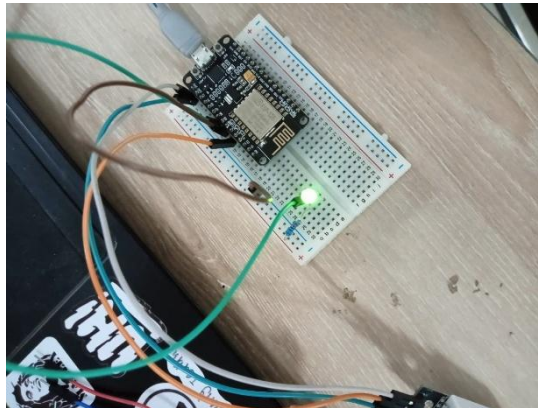
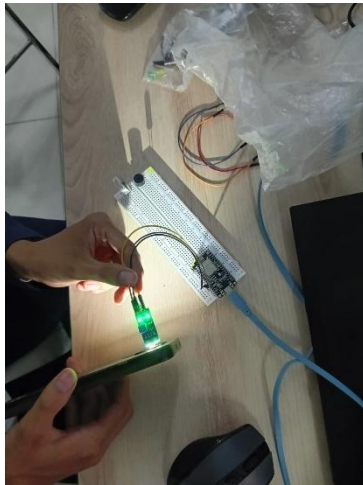


Pada Praktikum 1 ini, NodeMCU dikonfigurasi menjadi web server sederhana untuk menampilkan data suhu dari sensor DHT dan mengendalikan nyala LED melalui browser internet. Secara perangkat keras, sensor DHT dihubungkan ke pin D4 (GPIO 2), sedangkan lampu LED dipasang pada pin D6 (GPIO 12) dengan tambahan resistor 220 Ohm. Komponen-komponen ini dirakit di atas breadboard dan dihubungkan ke laptop menggunakan kabel USB, seperti yang terlihat pada dokumentasi file foto di sebelah kiri.

Dari sisi software, program dibuat menggunakan library ESP8266 Web Server agar berjalan di port 80. Antarmuka webnya ditulis langsung di dalam kode memakai Raw String Literal HTML. Halaman ini bersifat dinamis karena sistem akan otomatis mengganti teks %TEMPERATURE% di HTML dengan angka suhu asli dari sensor. Sistem juga dilengkapi routing URL; ketika tombol di web ditekan (URL /relay/on atau /relay/off), server akan mengubah status nyala LED dan otomatis me-refresh halaman web.

Berdasarkan hasil uji coba pada file foto di sebelah kanan, perangkat NodeMCU yang terdeteksi sebagai "ESP-F244E7" telah berhasil terkoneksi ke hotspot dari smartphone. Saat alamat IP server diakses lewat browser di laptop, antarmuka utama yang tertulis "ESP8266 Web Server" berhasil terbuka dan menampilkan suhu saat ini dengan nilai 2,00 Celcius. Selain itu, tombol hijau "ON" dan merah "OFF" di bagian Kontrol Relay sudah muncul dengan jelas di layar dan sudah bisa digunakan untuk mengatur rangkaian secara jarak jauh.

Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

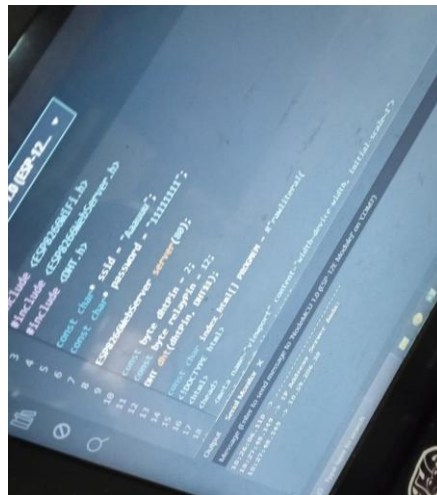


```
sketch_sep21b.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "Aaaa";
6 const char* password = "11111111";
7
8 const char* serverName = "http://10.59.206.30/relay/on";
9 const byte idPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("uClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   // ...
21 }
```

Output: Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM3]

Client Terhubung ke Wi-Fi!
HTTP Response code: 200
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303



Pada Praktikum 2 ini, NodeMCU dikonfigurasi sebagai HTTP Client yang bertugas mengirimkan perintah jarak jauh ke web server (NodeMCU dari Praktikum 1) melalui jaringan Wi-Fi. Secara perangkat keras, rangkaianannya cukup sederhana, yaitu menggunakan sensor cahaya LDR yang sudah dalam bentuk Modul board LDR jadi tidak memerlukan resistor cukup hubungkan pin VCC ke 3v3, GNG ke GNG dan jalur sinyalnya A0 dihubungkan ke pin analog A0 pada NodeMCU.

Dari sisi perangkat lunak, program memanfaatkan library ESP8266HTTPClient agar alat dapat mengeksekusi permintaan HTTP layaknya sebuah browser. Sistem membaca nilai LDR secara berkala. Ketika sensor tertutup atau gelap (nilai analog di bawah 300), NodeMCU secara otomatis bertindak mengirimkan perintah HTTP GET ke URL target milik server, yaitu pada rute /relay/on. Agar server tidak kewalahan menerima rentetan permintaan saat sensor terus-menerus gelap (flooding), program ini juga dilengkapi dengan delay atau jeda istirahat selama 10 detik setiap kali satu siklus pengiriman sukses dilakukan.

Berdasarkan hasil pengujian yang didokumentasikan pada file foto di atas skenario komunikasi antar-NodeMCU ini berhasil dilakukan. Pada foto kiri atas, terlihat pengujian fisik di mana sensor LDR sengaja di sinari Cahaya agar nilainya berubah lalu perubahan tersebut memicu alat untuk mengirimkan sinyal ke server, yang hasilnya langsung terlihat pada foto kanan atas berupa menyalanya lampu LED berwarna hijau di atas breadboard server. Keberhasilan komunikasi jaringan ini juga dikonfirmasi melalui 4ayer Arduino IDE di bagian bawah; Serial Monitor sukses menampilkan pesan “Client Terhubung ke Wi-Fi” yang disusul dengan munculnya log “HTTP Response code: 303”. Kode 303 tersebut menandakan bahwa request dari client telah sukses diterima oleh server dan berhasil di-redirect sesuai perintah di Praktikum 1.

Latihan

1. Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, dan bukan mode Access Point (AP)?

Dalam praktikum ini, mode Station dipilih karena memungkinkan NodeMCU sebagai Server dan Client berada dalam satu jaringan yang sama dengan perangkat pengguna, seperti handphone atau laptop. Dengan memakai router atau hotspot seluler yang sama, komunikasi antar perangkat IoT bisa berlangsung langsung tanpa perlu mengubah pengaturan routing dasar, dan juga memungkinkan perangkat tetap terhubung ke jaringan eksternal jika diperlukan. Sebaliknya, jika menggunakan mode Access Point, mikrokontroler akan memancarkan sinyal sendiri yang malah memisahkan jaringan lokal tersebut. Ini sangat mengganggu karena pengguna harus terus-menerus memutus dan menghubungkan kembali koneksi WiFi di laptop mereka hanya untuk mengakses antarmuka server NodeMCU, dan perangkat klien lainnya juga akan kesulitan untuk bergabung jika kapasitas batas koneksi AP pada ESP8266 sudah penuh.

2. Latihan 2 (Modifikasi Antarmuka & Telemetry): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada Web Server

- I. Menampilkan Kelembapan: Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman web.
- II. Tombol Dinamis Tunggal (Toggle Button): Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi LED saat ini:
 - a. Jika LED sedang OFF, tombol menampilkan tulisan "NYALAKAN (ON)" (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`
 - b. Jika LED sedang ON, tombol otomatis berubah menjadi tulisan "MATIKAN (OFF)" (warna merah) dan mengarahkan aksi ke `/relay/off`. (Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti

`%RELAY_BUTTON%`, lalu di fungsi `handleRoot()`, periksa status pin relay/LED menggunakan `digitalRead(relayPin)` sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai. Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah!

Jawaban

- a. Untuk menampilkan data kelembapan di halaman web, saya menambahkan sebuah paragraf baru yang berisi teks placeholder bernama `%HUMIDITY%` tepat di bawah baris indikator suhu di dalam blok `div sensor-box` pada struktur HTML yang masih mentah. Setelah antarmuka sudah siap, saya masuk ke dalam fungsi `handleRoot()` dan menambahkan deklarasi variabel baru untuk memanggil fungsi `readHumidity()` dari sensor DHT, sehingga program dapat membaca nilai kelembapan dari sensor tersebut. Nilai angka yang saya dapatkan selanjutnya dimasukkan ke dalam fungsi manipulasi HTML bernama `replace()` untuk mengganti teks `%HUMIDITY%` di dalam struktur web dengan nilai kelembapan yang sebenarnya secara otomatis sebelum halaman tersebut dikirim ke pengguna.
- b. Untuk membuat tombol kendali yang beradaptasi secara otomatis, langkah pertama saya adalah menghapus kedua kode tombol statis ON dan OFF dari kerangka HTML, lalu menggantinya dengan satu buah variabel placeholder bernama `%RELAY_BUTTON%`. Selanjutnya, di dalam fungsi `handleRoot()`, saya merancang sebuah algoritma percabangan `if-else` untuk mengecek status fisik dari pin LED menggunakan fungsi `digitalRead()`. Jika pembacaan mendeteksi pin sedang dalam kondisi HIGH atau menyala, maka program akan mengganti placeholder `%RELAY_BUTTON%` dengan sintaks HTML berisi tombol berwarna merah bertuliskan MATIKAN (OFF) yang menunjuk ke tautan `relay/off`. Sebaliknya, jika pin terdeteksi mati atau LOW, program akan menyiapkan sintaks HTML tombol berwarna hijau bertuliskan NYALAKAN (ON) yang mengarah ke tautan aksi `relay/on`.

Pada bagian penulisan kode kerangka HTML mentahnya, saya menyisipkan satu baris tag penampil tulisan yaitu

Kelembapan: `%HUMIDITY%` %

Di dalam kelas `sensor-box`, serta mengganti area link anchor tombol dengan teks `"%RELAY_BUTTON%"`. Sementara itu, untuk modifikasi kode di dalam blok fungsi `handleRoot()`, saya menambahkan baris kode

`"float h = dht.readHumidity();" dan baris`
`"html.replace("%HUMIDITY%", String(h));"` untuk mencetak kelembapan. Sedangkan untuk logika tombolnya, saya menulis baris kode dengan struktur `"if (digitalRead(relayPin) == HIGH) { html.replace("%RELAY_BUTTON%", "MATIKAN (OFF)"); } else { html.replace("%RELAY_BUTTON%", "NYALAKAN (ON)"); }"`. Seluruh potongan kode manipulasi string tersebut saya susun dan letakkan persis sebelum baris eksekusi akhir server titik send dipanggil.

3. Latihan 3 (Analisis Kode Client): Pada Praktikum 2, Serial Monitor Client mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh Serial Monitor Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (offline)? (Silakan cabut daya Server dan lihat hasil erornya).

Jika NodeMCU Server milik teman tiba-tiba dimatikan, layar Serial Monitor pada NodeMCU Client kita akan menampilkan pesan respons berupa kode respons HTTP dengan nilai negatif satu (-1). Angka negatif ini muncul karena NodeMCU Client secara aktif mencoba mengirimkan permintaan koneksi untuk protokol TCP ke alamat IP server di port delapan puluh, tetapi sayangnya server yang sudah tidak berfungsi tidak memberikan respons apa pun. Kondisi ini menyebabkan fungsi HTTP GET yang berjalan di memori perangkat klien mengalami batas waktu tunggu eksekusi atau timeout, sehingga pustaka bawaan ESP8266 secara otomatis menghentikan siklus tersebut dan mengembalikan kode error koneksi ditolak sebelum perangkat klien berhasil menerima teks status protokol HTTP standar.

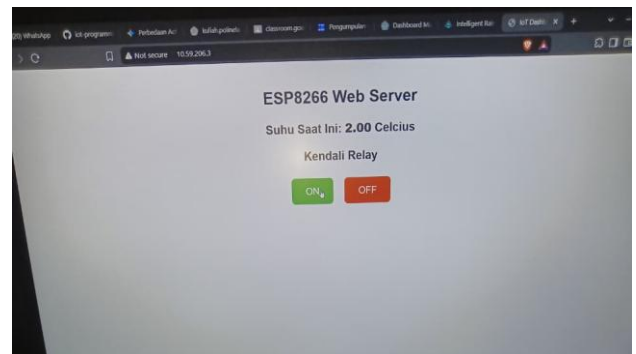
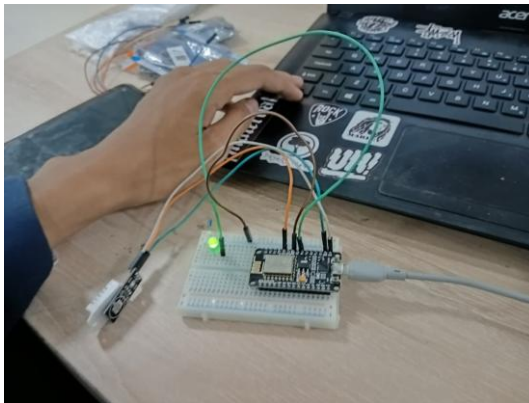
4. Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi conditional block if di Praktikum 2 sehingga Client mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Untuk menerapkan algoritma peringatan berdasarkan suhu, langkah pertama adalah menghilangkan bagian pembacaan nilai dari sensor LDR yang menggunakan pin analog, kemudian menggantinya dengan proses inisialisasi sensor DHT di awal program serta mendeklarasikan pin data yang terhubung ke kaki mikrokontroler. Dalam pengulangan fungsi `loop()`, saya menggunakan perintah `dht.readTemperature()` untuk memperoleh data suhu lingkungan sekitar dalam bentuk angka desimal. Intinya perubahan terletak pada cara penggunaan

fungsi if, di mana saya memeriksa dua kondisi yang mutlak menggunakan operator logika AND; kondisi pertama adalah memastikan fungsi pengecekan koneksi WiFi dalam keadaan WL_CONNECTED, dan kondisi kedua adalah memastikan nilai variabel pembacaan suhu lebih besar dari tiga puluh lima. Jika kedua syarat tersebut terpenuhi secara bersamaan, maka program diperbolehkan untuk menjalankan perintah HTTP begin guna mengarahkan ke alamat IP server dan mengirimkan permintaan metode GET. Setelah itu, harus diberikan jeda delay selama sekitar sepuluh detik untuk mencegah terjadinya overrequest atau terlalu banyak data yang masuk dan berpotensi merusak server teman.

Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur

```
</html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta http-equiv="refresh" content="5">
  <title>IoT Dashboard</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 50px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
    .btn-on { background-color: #4CAF50; color: white; border: none; }
    .btn-off { background-color: #f44336; color: white; border: none; }
    .sensor-box { font-size: 24px; font-weight: bold; }
  </style>
</head>
<body>
  <h1>ESP8266 Web Server</h1>
```



Dalam tugas pengembangan web server cerdas ini, program dari praktikum sebelumnya diubah sedemikian rupa sehingga antarmuka web dapat memperbarui data suhu secara otomatis tanpa harus menekan tombol refresh secara manual. Langkah ini dilakukan dengan memasukkan baris kode ke dalam bagian blok pada Raw Literal HTML, seperti yang terlihat dalam contoh kode pada file foto paling atas. Akibatnya, sistem berhasil mendorong peramban untuk memuat ulang halaman secara otomatis setiap 5 detik. Dengan demikian, data suhu yang diperbarui, seperti angka 2.00 Celcius yang muncul di antarmuka browser pada file foto di sebelah kanan bawah, dapat terus ditampilkan sesuai dengan kondisi NodeMCU yang sedang beroperasi.